

36660 Sport Analytics Project: NFL Referee Penalty Analysis

Kyle Berney, Erik Kvenlog, Jiashen Wang

2026-04-28

Contents

Load data	1
Process referee data	2
Team-game penalty data	2
EDA	3
Negative binomial crossed random effects model	5
Referee, offensive team, and defensive opponent effects	6
Bayesian crossed random effects model	9
Penalty-event data	14
Penalty type EDA	14
Penalty type mixed-effects models	17
Penalty type referee effects	18
Save processed outputs	20

```
library(tidyverse)
library(nflreadr)
library(janitor)
library(lme4)
library(glmmTMB)
library(brms)
library(broom.mixed)

theme_set(theme_minimal())
```

Load data

```
seasons = 2015:2025
season_type_keep = c("REG")

pbp_raw = load_pbp(seasons) |>
  clean_names()
```

```

officials_raw = load_officials(seasons) |>
  clean_names()

pbp = pbp_raw %>%
  filter(season %in% seasons) %>%
  filter(if ("season_type" %in% names(.)) season_type %in% season_type_keep else TRUE) %>%
  filter(!is.na(game_id))

officials = officials_raw %>%
  filter(season %in% seasons) %>%
  filter(if ("season_type" %in% names(.)) season_type %in% season_type_keep else TRUE)

```

Process referee data

```

referee_game = officials |>
  filter(position == "Referee") |>
  transmute(
    old_game_id = game_id,
    season,
    week,
    lead_referee = official_name,
    lead_ref_id = official_id
  )

```

Team-game penalty data

```

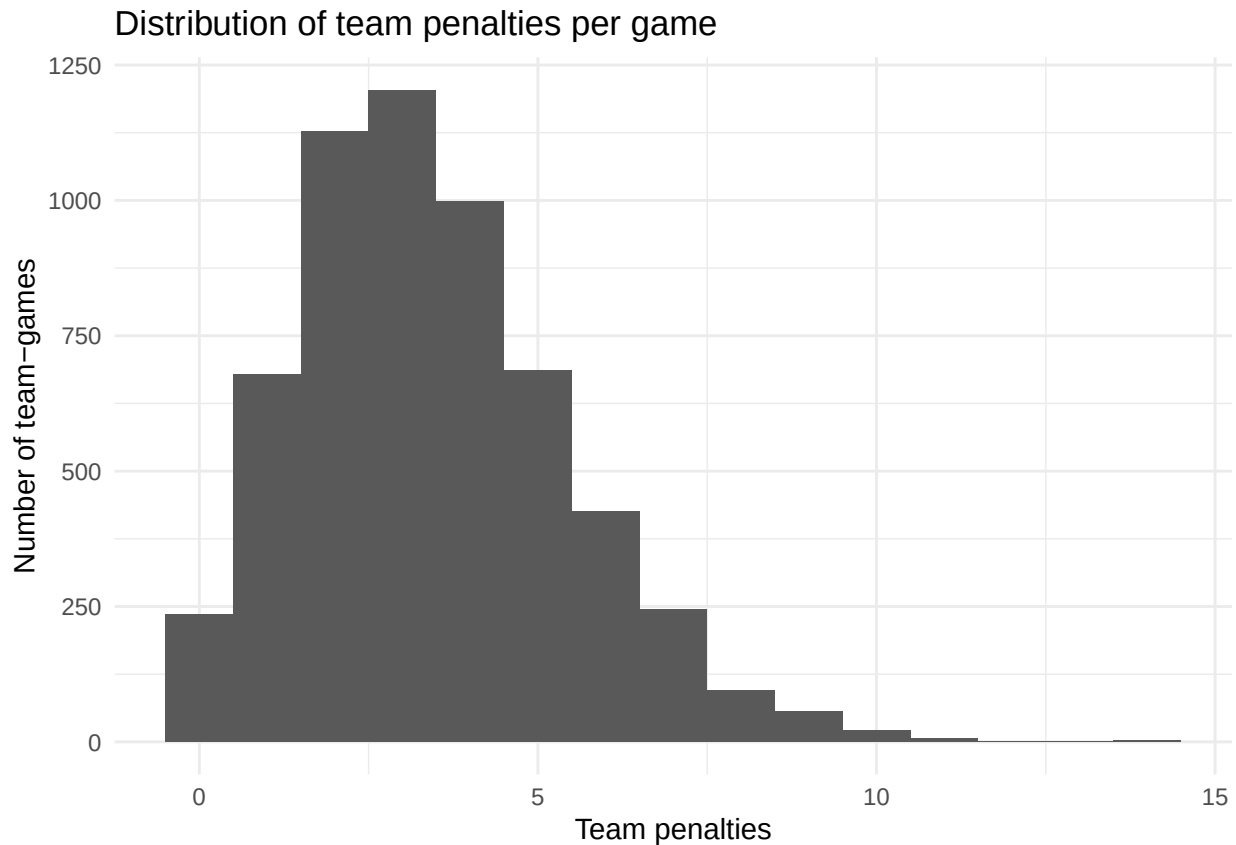
team_game_level = pbp |>
  filter(!is.na(posteam)) |>
  group_by(game_id, old_game_id, season, week, season_type,
    home_team, away_team, posteam, defteam) |>
  summarise(
    team_plays = n(),
    team_penalties = sum(penalty == 1 & penalty_team == first(posteam), na.rm = TRUE),
    team_penalty_yards = sum(if_else(penalty_team == first(posteam), penalty_yards, 0), na.rm = TRUE),
    offensive_penalty_rows = sum(penalty == 1 & posteam == penalty_team, na.rm = TRUE),
    mean_ydstogo = mean(ydstogo, na.rm = TRUE),
    mean_abs_score_diff = mean(abs(score_differential), na.rm = TRUE),
    early_down_rate = mean(down %in% c(1, 2), na.rm = TRUE),
    pass_rate = mean(play_type == "pass", na.rm = TRUE),
    rush_rate = mean(play_type == "run", na.rm = TRUE),
    is_home = as.integer(first(posteam) == first(home_team)),
    .groups = "drop"
  )

team_game_model_data = team_game_level |>
  left_join(referee_game, by = c("old_game_id", "season", "week")) |>
  filter(!is.na(lead_referee))

```

EDA

```
team_game_model_data |>
  ggplot(aes(x = team_penalties)) +
  geom_histogram(binwidth = 1) +
  labs(
    title = "Distribution of team penalties per game",
    x = "Team penalties",
    y = "Number of team-games"
  )
```



```
ref_raw_summary = team_game_model_data |>
  group_by(lead_referee) |>
  summarise(
    games = n(),
    mean_team_penalties = mean(team_penalties, na.rm = TRUE),
    mean_team_penalty_yards = mean(team_penalty_yards, na.rm = TRUE),
    .groups = "drop"
  ) |>
  filter(games >= 8) |>
  arrange(desc(mean_team_penalties))

ref_raw_summary |>
  slice_head(n = 15)
```

```
## # A tibble: 15 x 4
##   lead_referee  games mean_team_penalties mean_team_penalty_yards
```

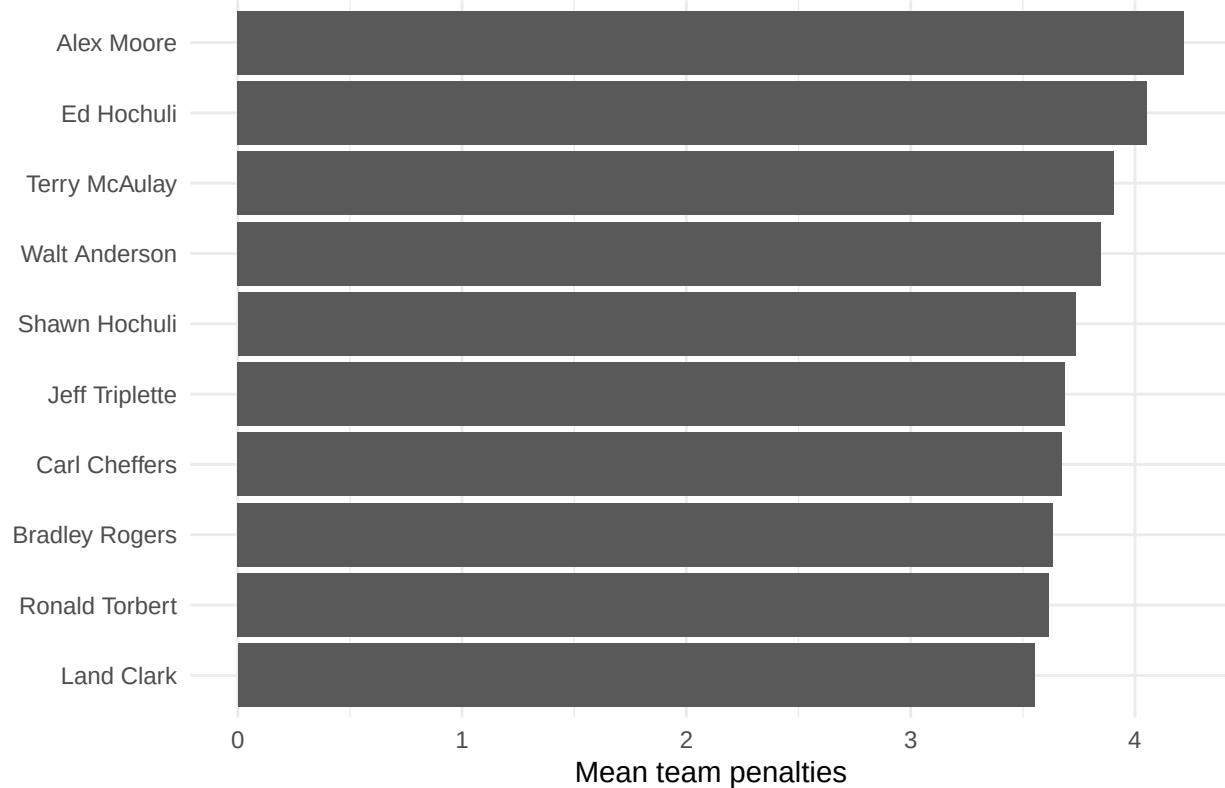
##	<chr>	<int>	<dbl>	<dbl>
##	1 Alex Moore	32	4.22	31.1
##	2 Ed Hochuli	92	4.05	33.5
##	3 Terry McAulay	86	3.91	31.8
##	4 Walt Anderson	152	3.85	30.2
##	5 Shawn Hochuli	248	3.73	29.3
##	6 Jeff Triplette	90	3.69	29.8
##	7 Carl Cheffers	340	3.67	28.6
##	8 Bradley Rogers	30	3.63	28.2
##	9 Ronald Torbert	342	3.62	27.9
##	10 Land Clark	190	3.55	26.9
##	11 Adrian Hill	218	3.53	26.3
##	12 Walt Coleman	124	3.52	26.9
##	13 Tony Corrente	214	3.51	26.6
##	14 Brad Rogers	186	3.49	26.5
##	15 Peter Morelli	120	3.49	27.4

```

ref_raw_summary |>
  slice_max(order_by = mean_team_penalties, n = 10) |>
  mutate(lead_referee = fct_reorder(lead_referee, mean_team_penalties)) |>
  ggplot(aes(x = lead_referee, y = mean_team_penalties)) +
  geom_col() +
  coord_flip() +
  labs(
    title = "Highest raw average team penalties by lead referee",
    x = NULL,
    y = "Mean team penalties"
  )

```

Highest raw average team penalties by lead referee



Negative binomial crossed random effects model

```
if (file.exists("models/nb_crossed_mod.rds")) {
  nb_crossed_mod = readRDS("models/nb_crossed_mod.rds")
} else {
  nb_crossed_mod = glmmTMB(
    team_penalties ~ offset(log(team_plays)) +
      is_home + mean_abs_score_diff + pass_rate + early_down_rate +
      (1 | lead_referee) + (1 | posteam) + (1 | defteam),
    family = nbinom2,
    data = team_game_model_data
  )

  saveRDS(nb_crossed_mod, "models/nb_crossed_mod.rds")
}
```

```
summary(nb_crossed_mod)

## Family: nbinom2 ( log )
## Formula:
## team_penalties ~ offset(log(team_plays)) + is_home + mean_abs_score_diff +
##   pass_rate + early_down_rate + (1 | lead_referee) + (1 | posteam) +
##   (1 | defteam)
## Data: team_game_model_data
##
##      AIC      BIC   logLik -2*log(L)  df.resid
```

```
##    23281.8    23341.8   -11631.9    23263.8        5775
##
## Random effects:
##
## Conditional model:
##   Groups      Name      Variance Std.Dev.
## lead_referee (Intercept) 0.0032025 0.05659
## posteam      (Intercept) 0.0011048 0.03324
## defteam      (Intercept) 0.0006537 0.02557
## Number of obs: 5784, groups: lead_referee, 29; posteam, 32; defteam, 32
##
## Dispersion parameter for nbinom2 family (): 114
##
## Conditional model:
##               Estimate Std. Error z value Pr(>|z|)
## (Intercept)    -2.649670   0.127002 -20.863 < 2e-16 ***
## is_home        -0.023972   0.014451  -1.659  0.09714 .
## mean_abs_score_diff -0.001643  0.001542  -1.066  0.28659
## pass_rate      -0.279096   0.088552  -3.152  0.00162 **
## early_down_rate -0.614617   0.189425  -3.245  0.00118 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Referee, offensive team, and defensive opponent effects

```
ref_effects = ranef(nb_crossed_mod)$cond$lead_referee |>
  as.data.frame() |>
  rownames_to_column("lead_referee") |>
  rename(ref_effect = `(Intercept)`)

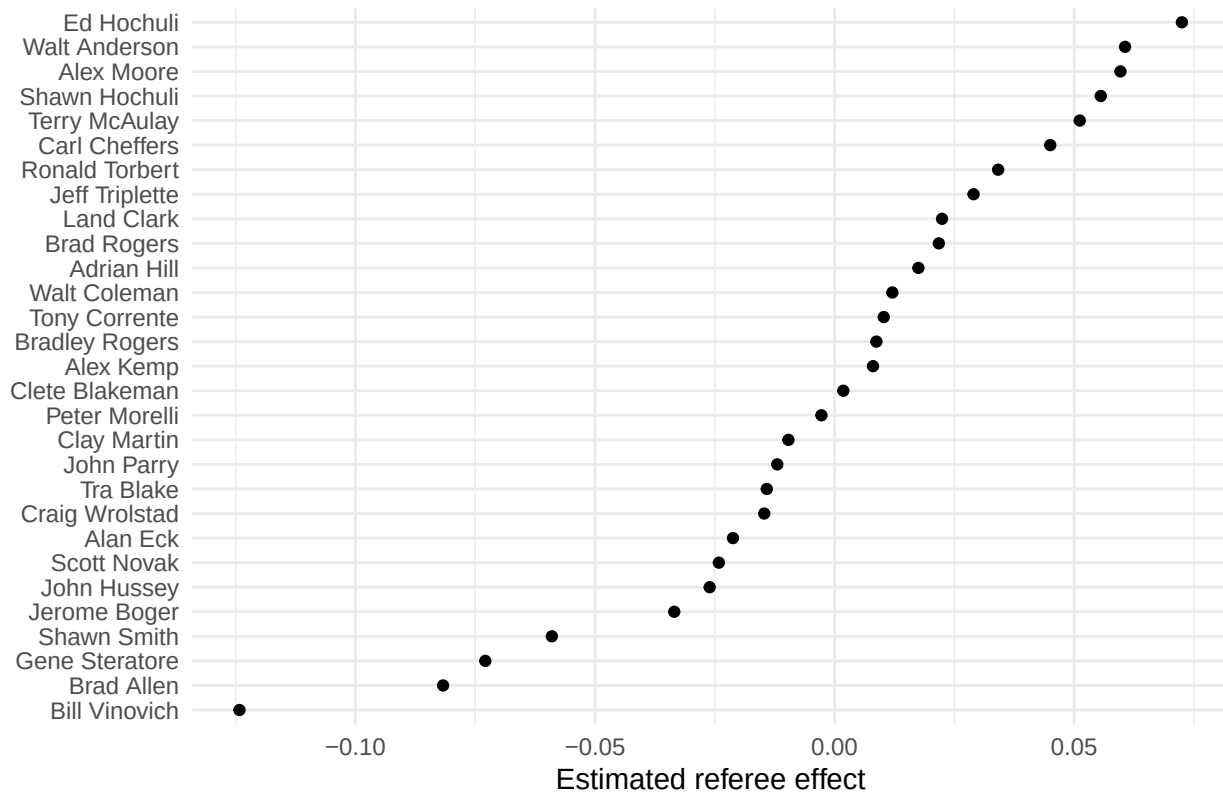
ref_effects |>
  arrange(desc(ref_effect)) |>
  slice_head(n = 10)

##      lead_referee ref_effect
## 1      Ed Hochuli 0.07248891
## 2    Walt Anderson 0.06062153
## 3      Alex Moore 0.05966038
## 4    Shawn Hochuli 0.05555578
## 5    Terry McAulay 0.05116606
## 6    Carl Cheffers 0.04500838
## 7   Ronald Torbert 0.03413224
## 8    Jeff Triplette 0.02899818
## 9      Land Clark 0.02241663
## 10   Brad Rogers 0.02175418

ref_effects |>
  mutate(lead_referee = fct_reorder(lead_referee, ref_effect)) |>
  ggplot(aes(x = lead_referee, y = ref_effect)) +
  geom_point() +
  coord_flip() +
  labs(
    title = "Lead referee random effects",
    x = NULL,
```

```
y = "Estimated referee effect"
)
```

Lead referee random effects



```
posteam_effects = ranef(nb_crossed_mod)$cond$posteam |>
  as.data.frame() |>
  rownames_to_column("posteam") |>
  rename(posteam_effect = `(Intercept)`)
```

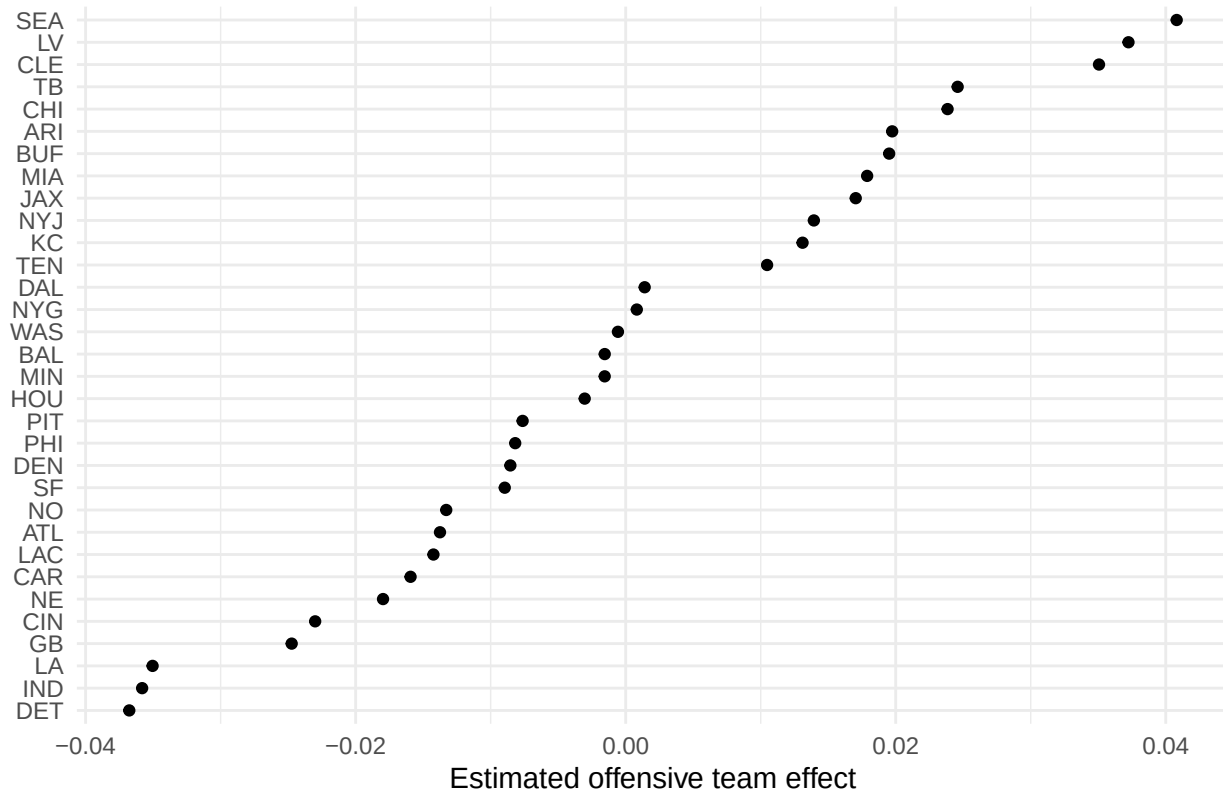
```
posteam_effects |>
  arrange(desc(posteam_effect)) |>
  slice_head(n = 10)
```

```
##   posteam posteam_effect
## 1     SEA      0.04080942
## 2     LV       0.03723849
## 3     CLE       0.03505694
## 4     TB       0.02458683
## 5     CHI       0.02384076
## 6     ARI       0.01973792
## 7     BUF       0.01951538
## 8     MIA       0.01788329
## 9     JAX       0.01703782
## 10    NYJ       0.01393803
```

```
posteam_effects |>
  mutate(posteam = fct_reorder(posteam, posteam_effect)) |>
  ggplot(aes(x = posteam, y = posteam_effect)) +
  geom_point() +
```

```
coord_flip() +
labs(
  title = "Offensive team random effects",
  x = NULL,
  y = "Estimated offensive team effect"
)
```

Offensive team random effects



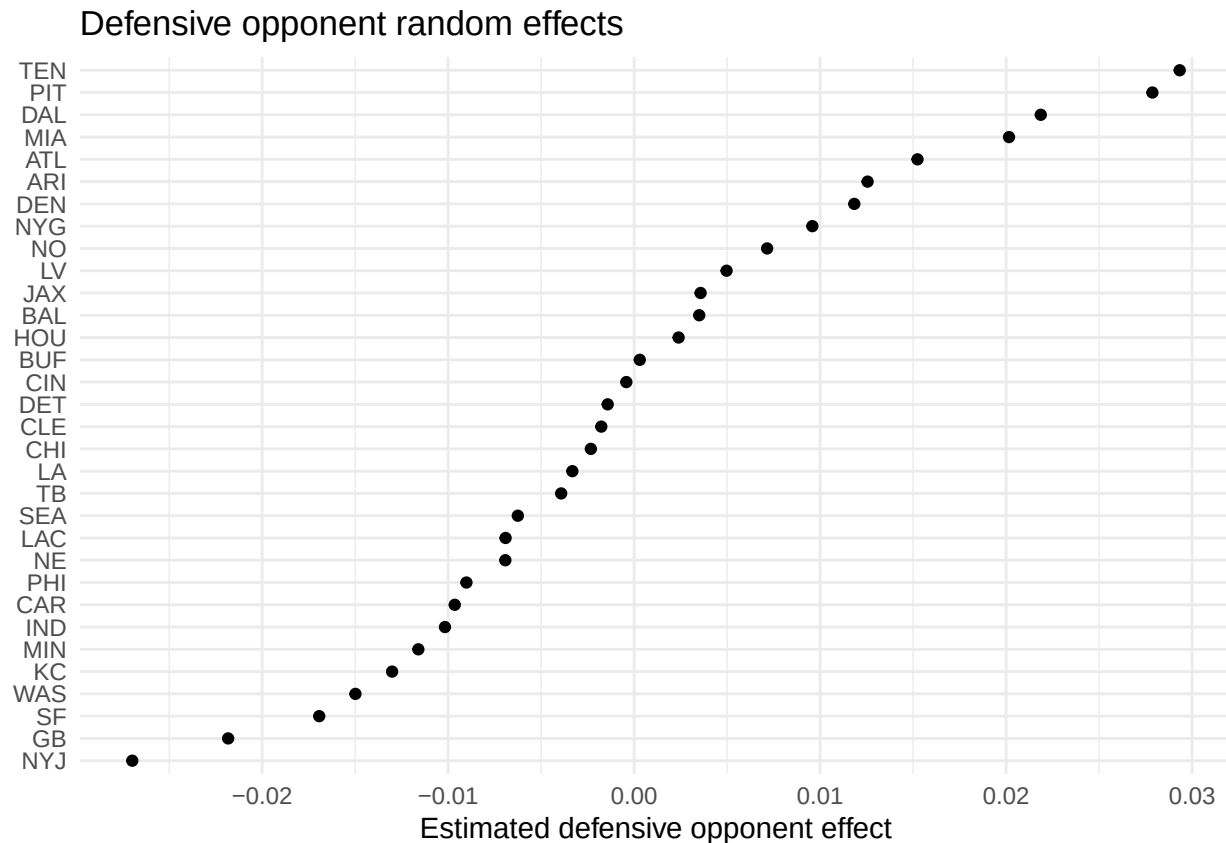
```
defteam_effects = ranef(nb_crossed_mod)$cond$defteam |>
  as.data.frame() |>
  rownames_to_column("defteam") |>
  rename(defteam_effect = `(Intercept)`)

defteam_effects |>
  arrange(desc(defteam_effect)) |>
  slice_head(n = 10)
```

```
##   defteam defteam_effect
## 1     TEN      0.029333862
## 2     PIT      0.027867333
## 3     DAL      0.021864877
## 4     MIA      0.020158485
## 5     ATL      0.015237261
## 6     ARI      0.012553293
## 7     DEN      0.011838427
## 8     NYG      0.009584422
## 9      NO      0.007151463
## 10    LV      0.004971191
```



```
defteam_effects |>
  mutate(defteam = fct_reorder(defteam, defteam_effect)) |>
  ggplot(aes(x = defteam, y = defteam_effect)) +
  geom_point() +
  coord_flip() +
  labs(
    title = "Defensive opponent random effects",
    x = NULL,
    y = "Estimated defensive opponent effect"
  )
)
```



Bayesian crossed random effects model

```
# This chunk can take a long time.
# After the first successful run, the fitted model is saved to models/bayes_crossed_mod.rds.
# For later runs, the saved model is loaded automatically.

if (file.exists("models/bayes_crossed_mod.rds")) {
  bayes_crossed_mod = readRDS("models/bayes_crossed_mod.rds")
} else {
  bayes_crossed_mod = brm(
    team_penalties ~ offset(log(team_plays)) +
      is_home + mean_abs_score_diff + pass_rate + early_down_rate +
      (1 | lead_referee) + (1 | posteam) + (1 | defteam),
    family = negbinomial(),
  )
}
```

```

data = team_game_model_data,
prior = c(
  prior(normal(0, 1), class = "b"),
  prior(normal(0, 2), class = "Intercept"),
  prior(student_t(3, 0, 2.5), class = "sd"),
  prior(student_t(3, 0, 2.5), class = "shape")
),
chains = 4,
iter = 2000,
cores = 10,
seed = 123
)

saveRDS(bayes_crossed_mod, "models/bayes_crossed_mod.rds")
}

summary(bayes_crossed_mod)

## Family: negbinomial
## Links: mu = log
## Formula: team_penalties ~ offset(log(team_plays)) + is_home + mean_abs_score_diff + pass_rate + early_
## Data: team_game_model_data (Number of observations: 5784)
## Draws: 4 chains, each with iter = 2000; warmup = 1000; thin = 1;
## total post-warmup draws = 4000
##
## Multilevel Hyperparameters:
## ~defteam (Number of levels: 32)
## Estimate Est.Error 1-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
## sd(Intercept) 0.02 0.01 0.00 0.05 1.01 797 967
##
## ~lead_referee (Number of levels: 29)
## Estimate Est.Error 1-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
## sd(Intercept) 0.06 0.01 0.04 0.09 1.00 1609 2395
##
## ~posteam (Number of levels: 32)
## Estimate Est.Error 1-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
## sd(Intercept) 0.03 0.01 0.01 0.06 1.00 969 692
##
## Regression Coefficients:
## Estimate Est.Error 1-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
## Intercept -2.67 0.13 -2.92 -2.42 1.00 5578 3057
## is_home -0.02 0.01 -0.05 0.00 1.00 6011 2753
## mean_abs_score_diff -0.00 0.00 -0.00 0.00 1.00 4496 3084
## pass_rate -0.28 0.09 -0.45 -0.10 1.00 6818 2748
## early_down_rate -0.59 0.19 -0.97 -0.21 1.00 5887 3186
##
## Further Distributional Parameters:
## Estimate Est.Error 1-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
## shape 80.32 39.77 40.92 182.15 1.00 3517 1884
##
## Draws were sampled using sampling(NUTS). For each parameter, Bulk_ESS
## and Tail_ESS are effective sample size measures, and Rhat is the potential
## scale reduction factor on split chains (at convergence, Rhat = 1).

```

```

bayes_ref_effects = ranef(bayes_crossed_mod)$lead_referee[, , "Intercept"] |>
  as.data.frame() |>
  rownames_to_column("lead_referee")

```

```

bayes_ref_effects |>
  arrange(desc(Estimate)) |>
  select(lead_referee, Estimate, Q2.5, Q97.5) |>
  slice_head(n = 10)

```

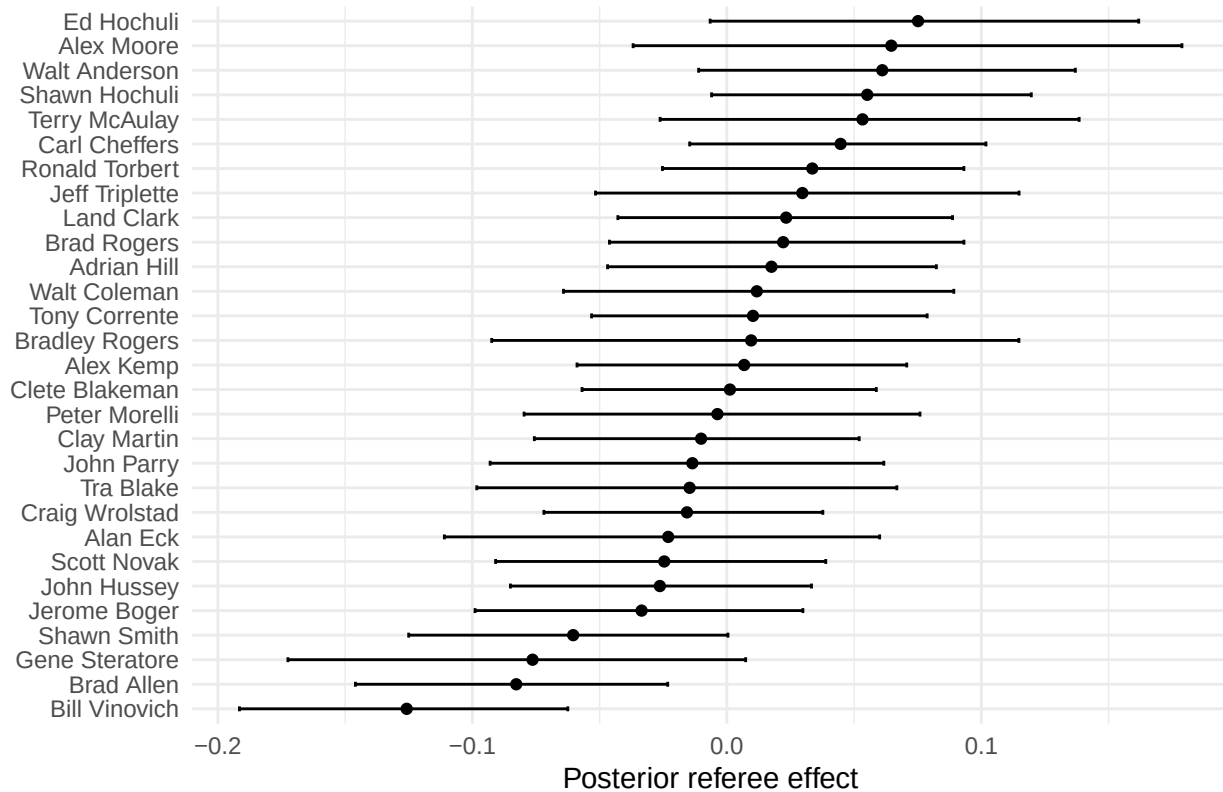
	lead_referee	Estimate	Q2.5	Q97.5
## 1	Ed Hochuli	0.07512778	-0.006573942	0.16183003
## 2	Alex Moore	0.06461069	-0.036843258	0.17881117
## 3	Walt Anderson	0.06106122	-0.011101116	0.13692692
## 4	Shawn Hochuli	0.05515478	-0.005999436	0.11962214
## 5	Terry McAulay	0.05331309	-0.026240274	0.13841824
## 6	Carl Cheffers	0.04469495	-0.014620850	0.10179144
## 7	Ronald Torbert	0.03356500	-0.025317369	0.09313278
## 8	Jeff Triplette	0.02965597	-0.051601919	0.11480917
## 9	Land Clark	0.02328643	-0.042829665	0.08866204
## 10	Brad Rogers	0.02212989	-0.046134290	0.09314969

```

bayes_ref_effects |>
  mutate(lead_referee = fct_reorder(lead_referee, Estimate)) |>
  ggplot(aes(x = lead_referee, y = Estimate)) +
  geom_point() +
  geom_errorbar(aes(ymin = Q2.5, ymax = Q97.5), width = 0.2) +
  coord_flip() +
  labs(
    title = "Posterior referee effects with 95% intervals",
    x = NULL,
    y = "Posterior referee effect"
  )

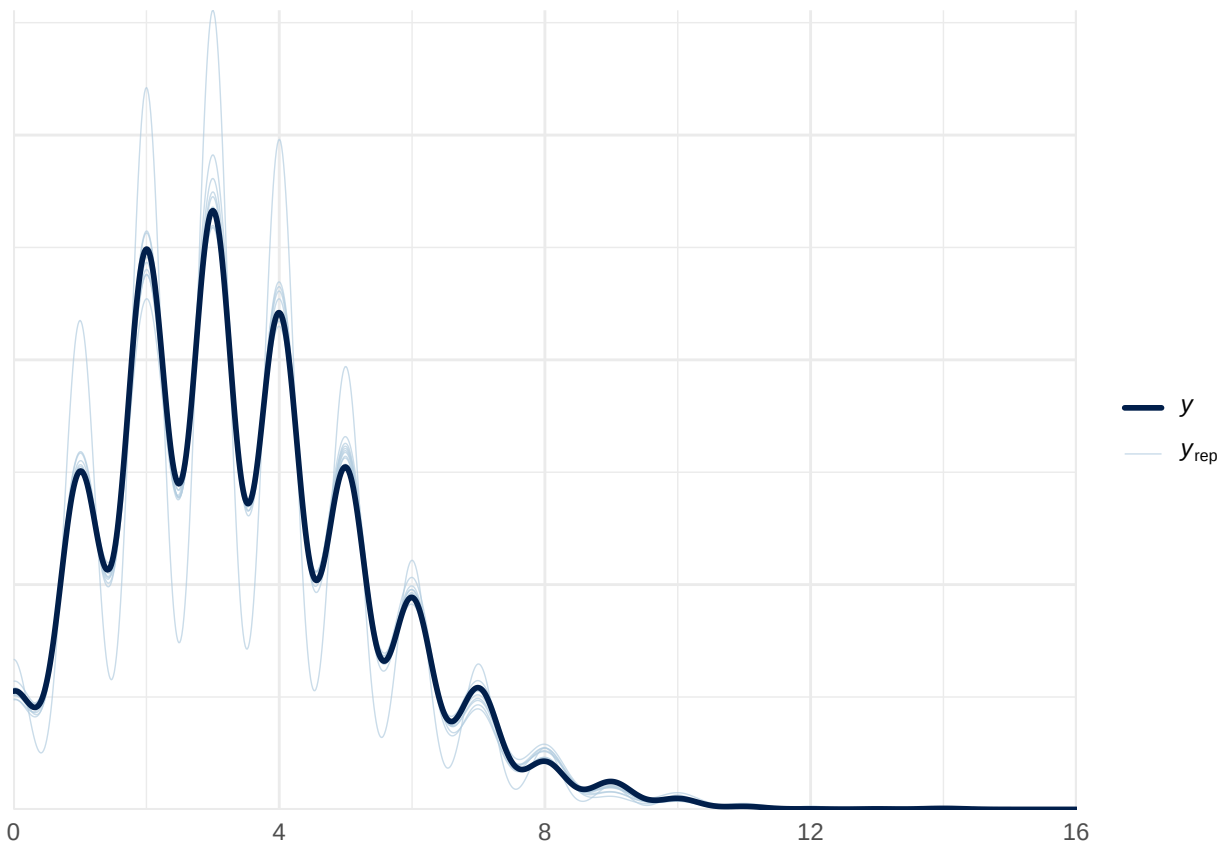
```

Posterior referee effects with 95% intervals



```
pp_check(bayes_crossed_mod)
```

```
## Using 10 posterior draws for ppc type 'dens_overlay' by default.
```



```
bayes_posteam_effects = ranef(bayes_crossed_mod)$posteam[, , "Intercept"] |>
  as.data.frame() |>
  rownames_to_column("posteam")
```

```
bayes_defteam_effects = ranef(bayes_crossed_mod)$defteam[, , "Intercept"] |>
  as.data.frame() |>
  rownames_to_column("defteam")
```

```
bayes_posteam_effects |>
  arrange(desc(Estimate)) |>
  select(posteam, Estimate, Q2.5, Q97.5) |>
  slice_head(n = 10)
```

##	posteam	Estimate	Q2.5	Q97.5
## 1	SEA	0.03827834	-0.01203537	0.10885183
## 2	LV	0.03370509	-0.01526998	0.09719286
## 3	CLE	0.03283523	-0.01519875	0.09683879
## 4	TB	0.02264325	-0.02441213	0.07960931
## 5	CHI	0.02185370	-0.02677892	0.08123754
## 6	ARI	0.01821598	-0.02742704	0.07249340
## 7	BUF	0.01739277	-0.03317431	0.07726980
## 8	MIA	0.01605208	-0.03341166	0.07344277
## 9	JAX	0.01590205	-0.03226683	0.07545041
## 10	NYJ	0.01255725	-0.03538250	0.07052173

```
bayes_defteam_effects |>
  arrange(desc(Estimate)) |>
  select(defteam, Estimate, Q2.5, Q97.5) |>
```

```
slice_head(n = 10)
```

```
##      defteam      Estimate      Q2.5      Q97.5
## 1      TEN 0.026508390 -0.01210665 0.08958268
## 2      PIT 0.025125361 -0.01550272 0.08485335
## 3      DAL 0.019713801 -0.01811547 0.07576906
## 4      MIA 0.018345767 -0.02004716 0.07393095
## 5      ATL 0.013572212 -0.02584427 0.06596679
## 6      ARI 0.011101995 -0.02902286 0.06199456
## 7      DEN 0.010586550 -0.02982227 0.05952122
## 8      NYG 0.008698535 -0.03200070 0.05743144
## 9      NO  0.006927454 -0.03498343 0.05523759
## 10     LV  0.004383795 -0.03677255 0.04901729
```

Penalty-event data

```
penalty_events_model_data = pbp |>
  filter(penalty == 1, !is.na(penalty_type), !is.na(posteam)) |>
  group_by(old_game_id, season, week) |>
  mutate(
    mean_abs_score_diff = mean(abs(score_differential), na.rm = TRUE),
    pass_rate = mean(play_type == "pass", na.rm = TRUE),
    early_down_rate = mean(down %in% c(1, 2), na.rm = TRUE)
  ) |>
  ungroup() |>
  transmute(
    game_id,
    old_game_id,
    season,
    week,
    posteam,
    defteam,
    penalty_type,
    is_home = as.integer(posteam == home_team),
    mean_abs_score_diff,
    pass_rate,
    early_down_rate
  ) |>
  left_join(referee_game, by = c("old_game_id", "season", "week")) |>
  filter(!is.na(lead_referee))
```

Penalty type EDA

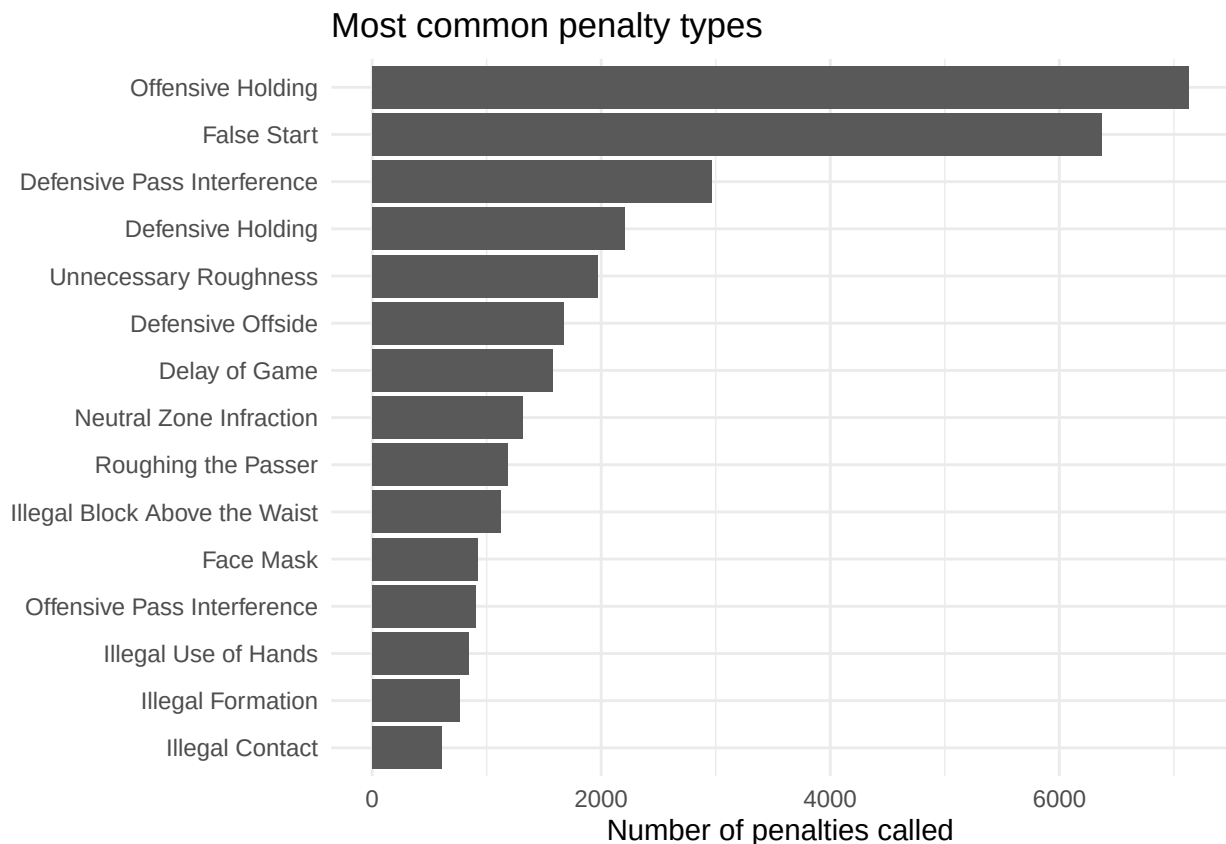
```
penalty_type_counts = penalty_events_model_data |>
  count(penalty_type, sort = TRUE)

penalty_type_counts |>
  slice_head(n = 15)
```

```
## # A tibble: 15 x 2
##   penalty_type      n
##   <chr>          <int>
```

```
## 1 Offensive Holding          7128
## 2 False Start                6372
## 3 Defensive Pass Interference 2967
## 4 Defensive Holding          2206
## 5 Unnecessary Roughness      1970
## 6 Defensive Offside          1675
## 7 Delay of Game              1580
## 8 Neutral Zone Infraction    1316
## 9 Roughing the Passer        1184
## 10 Illegal Block Above the Waist 1119
## 11 Face Mask                  919
## 12 Offensive Pass Interference 903
## 13 Illegal Use of Hands       841
## 14 Illegal Formation          761
## 15 Illegal Contact           603
```

```
penalty_type_counts |>
  slice_head(n = 15) |>
  mutate(penalty_type = fct_reorder(penalty_type, n)) |>
  ggplot(aes(x = penalty_type, y = n)) +
  geom_col() +
  coord_flip() +
  labs(
    title = "Most common penalty types",
    x = NULL,
    y = "Number of penalties called"
  )
```



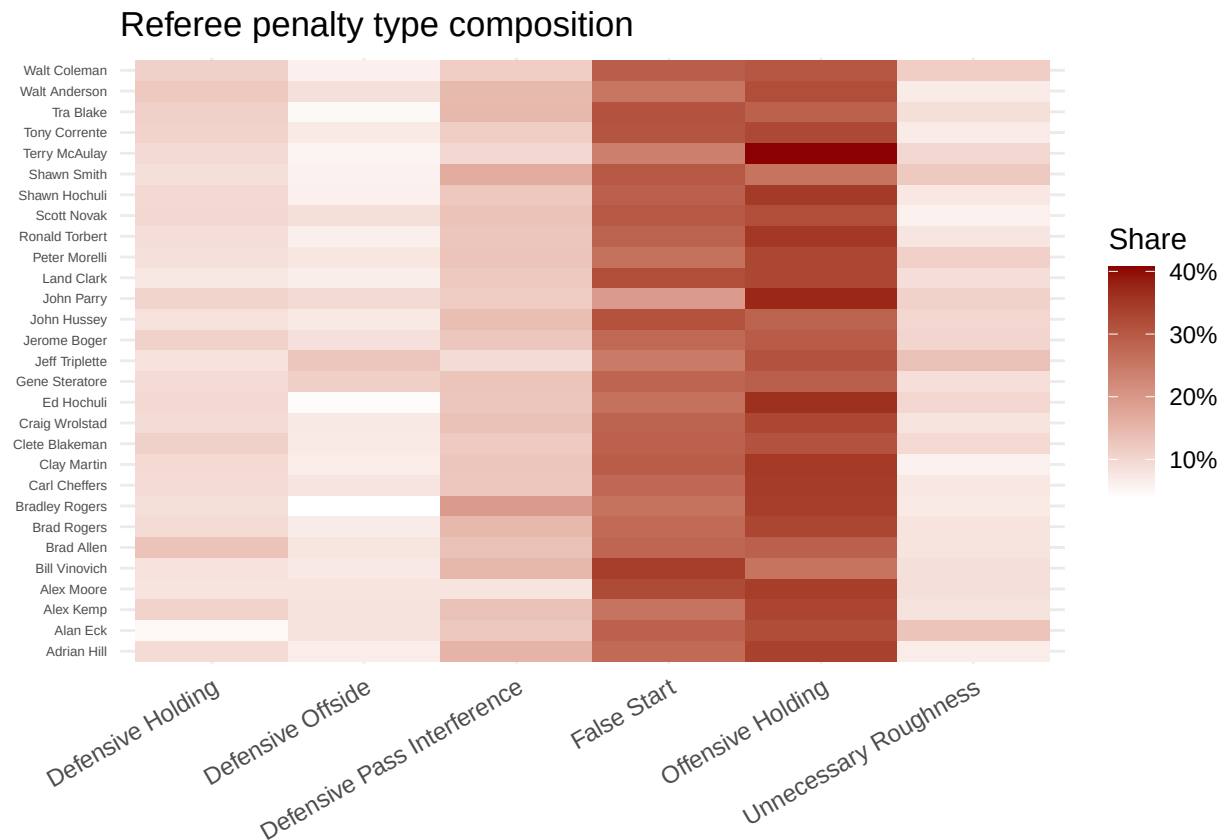
```

top_penalty_types = penalty_type_counts |>
  slice_head(n = 6) |>
  pull(penalty_type)

ref_type_heatmap = penalty_events_model_data |>
  filter(penalty_type %in% top_penalty_types) |>
  group_by(lead_referee, penalty_type) |>
  summarise(n = n(), .groups = "drop") |>
  group_by(lead_referee) |>
  mutate(pct = n / sum(n)) |>
  filter(sum(n) >= 50) |>
  ungroup()

ref_type_heatmap |>
  ggplot(aes(x = penalty_type, y = lead_referee, fill = pct)) +
  geom_tile() +
  scale_fill_gradient(low = "white", high = "darkred", labels = scales::percent) +
  labs(
    title = "Referee penalty type composition",
    x = NULL,
    y = NULL,
    fill = "Share"
  ) +
  theme(
    axis.text.x = element_text(angle = 30, hjust = 1),
    axis.text.y = element_text(size = 5)
  )

```



Penalty type mixed-effects models

```
penalty_events_model_data = penalty_events_model_data |>
  mutate(
    is_offensive_holding = as.integer(penalty_type == "Offensive Holding"),
    is_false_start = as.integer(penalty_type == "False Start"),
    is_defensive_pass_interference = as.integer(penalty_type == "Defensive Pass Interference"),
    is_defensive_holding = as.integer(penalty_type == "Defensive Holding"),
    is_unnecessary_roughness = as.integer(penalty_type == "Unnecessary Roughness"),
    is_defensive_offside = as.integer(penalty_type == "Defensive Offside")
  )

# These models may take some time.
# After the first successful run, the fitted model list is saved to models/penalty_type_models.rds.
# For later runs, the saved model list is loaded automatically.

if (file.exists("models/penalty_type_models.rds")) {
  penalty_type_models = readRDS("models/penalty_type_models.rds")
} else {
  offensive_holding_mod = glmer(
    is_offensive_holding ~ is_home + mean_abs_score_diff + pass_rate + early_down_rate +
      (1 | lead_referee) + (1 | posteam) + (1 | defteam),
    family = binomial,
    data = penalty_events_model_data
  )

  false_start_mod = glmer(
    is_false_start ~ is_home + mean_abs_score_diff + pass_rate + early_down_rate +
      (1 | lead_referee) + (1 | posteam) + (1 | defteam),
    family = binomial,
    data = penalty_events_model_data
  )

  defensive_pass_interference_mod = glmer(
    is_defensive_pass_interference ~ is_home + mean_abs_score_diff + pass_rate + early_down_rate +
      (1 | lead_referee) + (1 | posteam) + (1 | defteam),
    family = binomial,
    data = penalty_events_model_data
  )

  defensive_holding_mod = glmer(
    is_defensive_holding ~ is_home + mean_abs_score_diff + pass_rate + early_down_rate +
      (1 | lead_referee) + (1 | posteam) + (1 | defteam),
    family = binomial,
    data = penalty_events_model_data
  )

  unnecessary_roughness_mod = glmer(
    is_unnecessary_roughness ~ is_home + mean_abs_score_diff + pass_rate + early_down_rate +
      (1 | lead_referee) + (1 | posteam) + (1 | defteam),
    family = binomial,
    data = penalty_events_model_data
  )
}
```

```

defensive_offside_mod = glmer(
  is_defensive_offside ~ is_home + mean_abs_score_diff + pass_rate + early_down_rate +
    (1 | lead_referee) + (1 | posteam) + (1 | defteam),
  family = binomial,
  data = penalty_events_model_data
)

penalty_type_models = list(
  "Offensive Holding" = offensive_holding_mod,
  "False Start" = false_start_mod,
  "Defensive Pass Interference" = defensive_pass_interference_mod,
  "Defensive Holding" = defensive_holding_mod,
  "Unnecessary Roughness" = unnecessary_roughness_mod,
  "Defensive Offside" = defensive_offside_mod
)

saveRDS(penalty_type_models, "models/penalty_type_models.rds")
}

```

Penalty type referee effects

```

extract_ref_effects = function(model, penalty_label) {
  ranef(model)$lead_referee |>
    as.data.frame() |>
    rownames_to_column("lead_referee") |>
    rename(ref_effect = `(Intercept)`) |>
    mutate(penalty_type = penalty_label)
}

all_ref_effects = bind_rows(
  extract_ref_effects(penalty_type_models[["Offensive Holding"]], "Offensive Holding"),
  extract_ref_effects(penalty_type_models[["False Start"]], "False Start"),
  extract_ref_effects(penalty_type_models[["Defensive Pass Interference"]], "Defensive Pass Interference"),
  extract_ref_effects(penalty_type_models[["Defensive Holding"]], "Defensive Holding"),
  extract_ref_effects(penalty_type_models[["Unnecessary Roughness"]], "Unnecessary Roughness"),
  extract_ref_effects(penalty_type_models[["Defensive Offside"]], "Defensive Offside")
)

all_ref_effects |>
  arrange(penalty_type, desc(ref_effect)) |>
  group_by(penalty_type) |>
  slice_head(n = 5)

## # A tibble: 30 x 3
## # Groups:   penalty_type [6]
##   lead_referee   ref_effect penalty_type
##   <chr>          <dbl> <chr>
## 1 Brad Allen      0.174 Defensive Holding
## 2 Clete Blakeman  0.0754 Defensive Holding
## 3 Walt Anderson   0.0682 Defensive Holding
## 4 Jerome Boger    0.0507 Defensive Holding
## 5 Tony Corrente   0.0441 Defensive Holding
## 6 Jeff Triplette  0.219  Defensive Offside

```

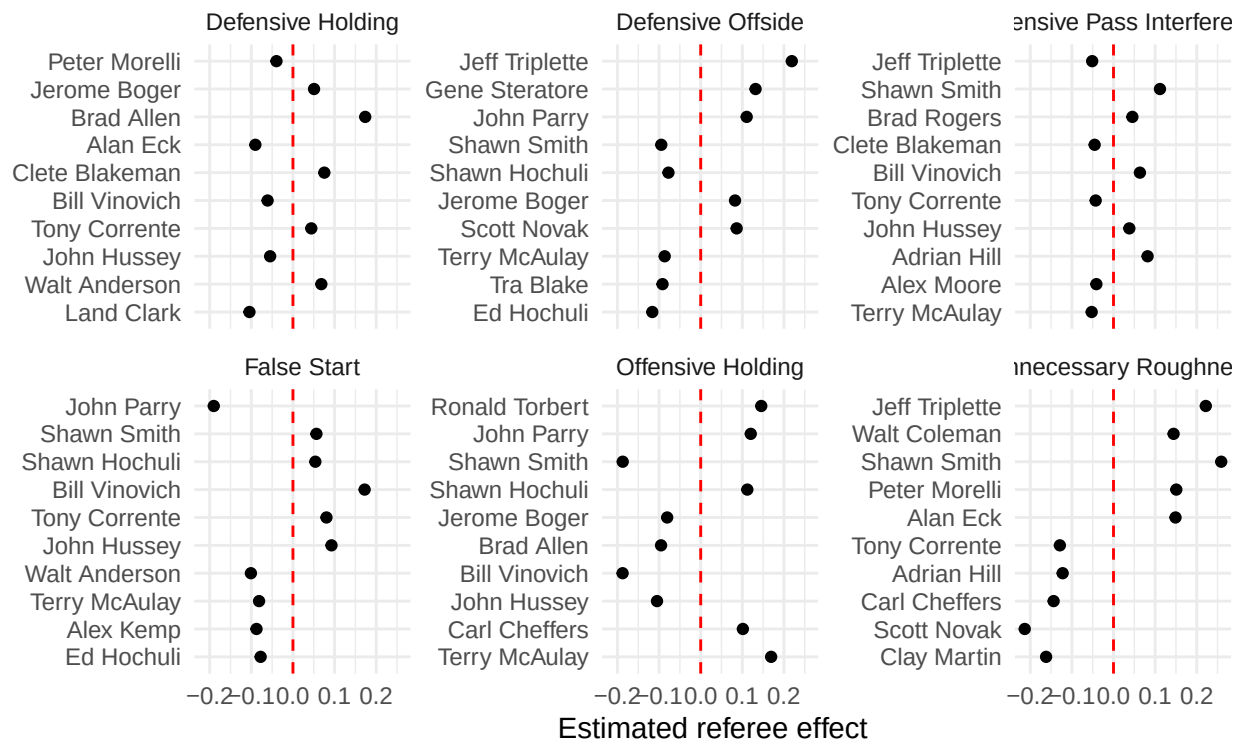
```
## 7 Gene Steratore      0.132 Defensive Offside
## 8 John Parry          0.111 Defensive Offside
## 9 Scott Novak         0.0864 Defensive Offside
## 10 Jerome Boger       0.0825 Defensive Offside
## # i 20 more rows
```

```
top_bottom_ref_effects = all_ref_effects |>
  group_by(penalty_type) |>
  slice_max(order_by = ref_effect, n = 5) |>
  bind_rows(
    all_ref_effects |>
      group_by(penalty_type) |>
      slice_min(order_by = ref_effect, n = 5)
  ) |>
  ungroup()

top_bottom_ref_effects |>
  mutate(lead_referee = fct_reorder(lead_referee, ref_effect)) |>
  ggplot(aes(x = lead_referee, y = ref_effect)) +
  geom_point(size = 1.5) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "red") +
  coord_flip() +
  facet_wrap(~ penalty_type, nrow = 2, scales = "free_y") +
  labs(
    title = "Referee random effects by penalty type",
    subtitle = "Top and bottom 5 referees per penalty type",
    x = NULL,
    y = "Estimated referee effect"
  )
```

Referee random effects by penalty type

Top and bottom 5 referees per penalty type



```
model_aic = AIC(
  penalty_type_models[["Offensive Holding"]],
  penalty_type_models[["False Start"]],
  penalty_type_models[["Defensive Pass Interference"]],
  penalty_type_models[["Defensive Holding"]],
  penalty_type_models[["Unnecessary Roughness"]],
  penalty_type_models[["Defensive Offside"]]
) |>
  rownames_to_column("model") |>
  mutate(model = names(penalty_type_models)) |>
  arrange(AIC)
```

model_aic

```
##           model df      AIC
## 1    Defensive Offside  8 13511.89
## 2    Unnecessary Roughness  8 15085.11
## 3    Defensive Holding  8 16596.38
## 4 Defensive Pass Interference  8 20492.58
## 5           False Start  8 33685.49
## 6    Offensive Holding  8 35934.77
```

Save processed outputs

```
saveRDS(team_game_model_data, "models/team_game_model_data.rds")
saveRDS(penalty_events_model_data, "models/penalty_events_model_data.rds")
```

```
saveRDS(ref_effects, "models/ref_effects.rds")
saveRDS(postteam_effects, "models/postteam_effects.rds")
saveRDS(defteam_effects, "models/defteam_effects.rds")
saveRDS(all_ref_effects, "models/penalty_type_ref_effects.rds")
```